

Assignment 1

Date: 6/8/'26

1. Accept an integer and determine whether it is:
Armstrong Number
Palindrome Number
Perfect Number
Display every property satisfied.
2. Implement the Euclidean Algorithm to compute the GCD of two integers. Then compute the LCM using the computed GCD.
3. The following program produces incorrect results. Explain why.

```
x=input()
print(x+5)
```
4. Find all indentation errors and correct them.
 - a.

```
for i in range(5):
    print(i)
    if i==3:
        break
```
 - b.

```
def area(r):
    pi=3.14
    return pi*r*r
```

Operators — in, not in, is, is not

5. Given `L = [10, 20, 30, [40, 50]]`, write code to check whether 40 is a direct member of L (should print False since it's nested) and whether it exists at all if you flatten one level.
6. Explain with code why the following behaves differently for small and large integers:

```
a = 5
b = 5
print(a is b)
```

```
a = 500
b = 500
print(a is b)
```

Write a program that demonstrates this using `id()`, and note the range of integers Python caches (-5 to 256).

7. Take a string `s = "hello"` and `t = "hello"`. **Check `[ s is t ]` vs `[ s == t ]`.** Then do the same with `s = "hello world!"` and `t = "hello world!"` (a string with a space/special char) and observe the difference. Explain interning in your own words in a comment.

8. Write a program that takes a list of numbers and a target value, and uses **in/not in** to classify which target values exist and which don't — without using `if x in list: print(...)` directly; instead, build a small membership-report function.

Dangling-Else Problem - Python avoids the classic dangling-else ambiguity (like in C) because of forced indentation. Write a short program with nested **if** statements that would be ambiguous in C-like pseudocode, and show how Python's indentation resolves it unambiguously. Include a comment showing what the "wrong" C-style interpretation would have been.

9. Trace through with `a = -1, b = 5` and explain (in a comment) which if the else binds to and why there's no ambiguity in Python.
 - i. Comments: `if a > 0: if b > 0: print("both positive") else: print("a not positive")`

Ternary Operator

10. WAP for one-line ternary expression to find the maximum of three numbers a, b, c (nested ternary allowed)
11. Using a ternary operator, classify a number as "Positive", "Negative", or "Zero" — think about whether you can do 3-way classification cleanly with a single ternary, or whether it needs nesting. Comment on readability tradeoffs.

While, While-else, For, For-else

12. WAP to find the sum of a number until it becomes a single digit. (example: `9875 → 29 → 11 → 2`)
13. WAP to search for a target element in a list. DO NOT use a separate flag variable -rely purely on **'while-else'** semantics (loop completed without break)
14. WAP using **'for-else'** to check if a list contains any duplicate elements. Print "No duplicates found" if the loop runs to completion.

Break, Continue, Pass

15. WAP to print all numbers from 1 to 50, skipping multiples of 5- where `print("multiple of 5")` for `(n<=20)`, for multiples of 3 within 1-20, use `pass` as a placeholder (no-op) to prove `pass` does not skip printing the number afterward, and stopping completely once you hit a number greater than 40.
16. Generate a secret number inside the program. Keep asking the user until the correct number is entered. Display the number of attempts.

Patterns

17. WAP to print Floyd's Triangle numbers up to n rows.

18. WAP to print the following pattern for n numbers.

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

19. WAP for: A

A B

A B C

A B C D

20. WAP for:

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```